🔥 CampFire-1: Kerberoasting Attack Analysis — Beginner's Guide

> This guide explains everything from ZERO. No prior knowledge needed. We use simple examples first, then solve the real challenge step-by-step.

Table of Contents

What is this challenge about?

Someone attacked a company's computer network. We are like **detectives** 🕵️. We are given **evidence** (log files) and we must figure out:

- What happened?
- Who did it?
- Which tools did they use?
- When did it happen?

This is called **Digital Forensics** — just like crime scene investigation, but for computers.

Basic Concept 1: Active Directory

Simple example first:

Imagine a big school 🏫 .The school has:

- A **register/database** with every student's name, class, and ID card.
- Rules like "only teachers can enter the staff room."
- A **main office** that checks everyone's ID before giving access.

Active Directory (AD) is exactly this, but for a company's computer network.

- It stores all **usernames**, **passwords (hashed)**, **computers**, and **permissions**.
- It decides "who can access what."
- Companies use it so employees can log in to any office computer with one username/password.

In our case: The company is called `FORELA.LOCAL` (like a school name).

Basic Concept 2: Domain Controller

Simple example:

If Active Directory is the "register book," the **Domain Controller (DC)** is the **office/room** where that register book is kept and managed.

- It's a special server (computer) that holds the Active Directory database.
- Every time someone logs in, the Domain Controller checks: "Is this password correct?"
- It also keeps **logs** (a diary) of everything that happens — every login, every request.

In our case: The server is named `DC01`.

Basic Concept 3: Kerberos

Real-life example first:

Imagine an amusement park 🎢 :

1. You go to the **ticket counter** and show your ID. They give you a **wristband** (proof you paid).
2. Now, to enter any ride, you just show your **wristband** — you don't need to show your ID again and again.
3. Each ride checks the wristband and lets you in.

3. Each ride checks the wristband and lets you in.

Kerberos is a login system that works the same way for computers:

Amusement Park	Kerberos Term
ID card	Username + Password
Ticket counter	Domain Controller (Key Distribution Center)
Wristband	TGT (Ticket Granting Ticket)
Ride	A Service (like a database, file server)
Ride-specific pass	TGS (Ticket Granting Service) ticket

Flow:

1. User logs in with password → gets a **TGT** (like the wristband).
2. User wants to use a **service** (example: a SQL database) → shows TGT → gets a **TGS ticket** specific to that service.
3. User gives the TGS ticket to the service → service allows access.

The TGS ticket is **encrypted** using the **password hash of the service account** (the account that runs that service, like "MSSQLService").

Basic Concept 4: Kerberoasting

Simple example:

Imagine a bad visitor at the amusement park who:

1. Politely asks the ticket counter for a "ride pass" for EVERY ride — even rides he doesn't plan to use.
2. Takes those passes home.
3. Uses a supercomputer to try millions of combinations to **crack open** the pass and reveal the ticket counter's secret stamp (which is basically a password).
4. If he cracks it, he now has the **service's password** and can pretend to be that service anytime.

This is Kerberoasting.

Technical version:

1. Attacker (already has SOME normal user account) asks the Domain Controller for TGS tickets for **service accounts** (like MSSQLService).
2. Any authenticated user is ALLOWED to request these tickets — this is normal Kerberos

behavior, not a "hack" by itself.

3. These TGS tickets are encrypted with the **service account's password hash**.
4. Attacker takes the ticket **offline** and tries to crack the password hash using tools (like Hashcat).
5. If the service account has a weak password, attacker cracks it and now has valid credentials for that service account — often with high privileges.

Why RC4 (0x17) matters:

- Kerberos tickets can be encrypted with different algorithms.
- `RC4 (0x17)` is an **old, weak** encryption. It's much easier and faster to crack than modern `AES` encryption.
- So, attackers **force** the Domain Controller to give RC4-encrypted tickets, because those are easier to crack.
- Seeing `0x17` in logs is a **red flag** for Kerberoasting.

Basic Concept 5: Windows Event Logs

Simple example:

Think of a **CCTV camera with a notebook** 📓 next to every door in a building. Every time someone opens a door, the notebook writes:

- Who opened it
- What time
- Which door

Windows Event Logs work the same way for a computer. Windows writes down every important action:

- Logins
- Password changes
- Services started
- Scripts run

These logs are stored in `.evtx` files and viewed using a tool called **Event Viewer**.

Types of logs we use here:

- **Security Logs** (from Domain Controller) → records logins, ticket requests, etc.
- **PowerShell Operational Logs** (from a normal computer/workstation) → records PowerShell script activity.

Basic Concept 6: Event ID

Simple example:

If the notebook (event log) has thousands of entries, how do we quickly find "all door-opening events" and ignore "all light-switch events"?

Windows gives every **type of action** a unique **number**, called **Event ID**. It's like a **category code**.

Important Event IDs in this case:

Event ID	Meaning
`4769`	A Kerberos Service Ticket was requested (this is the key event for Kerberoasting)
`4771`	Kerberos pre-authentication failed
`4104`	A PowerShell script block was executed (shows full script content)

So instead of reading 1 million logs, we **filter by Event ID** — just like searching "show me only door-opening entries."

Basic Concept 7: Prefetch Files

Simple example:

Imagine your mom notices which snacks you eat often, so she keeps them near the front of the fridge for you to grab quickly next time. She keeps a "note" of:

- Which snack
- How many times you ate it
- When was the last time

Windows Prefetch does the same thing for **programs (.exe files)**:

- Whenever you run a program, Windows creates a small file recording:
  - Program name
  - Number of times it ran
  - **Last run time** & **previous run times**
  - Files it touched while running (DLLs, paths, etc.)

This is stored in: `C:\Windows\Prefetch\*.pf`

Why forensic investigators love this: Even if the attacker deletes the tool (like `Rubeus.exe`) afterward, the Prefetch file **still proves** it was executed, when, and from where (the file path!).

Basic Concept 8: PowerShell Logging

Simple example:

PowerShell is like a "command box" in Windows where you can type instructions to control the computer (similar to a command terminal).

Attackers love PowerShell because:

- It's already installed on every Windows computer (no need to bring extra tools).
- It's powerful — can enumerate users, dump passwords, download files, etc.

PowerShell Operational Logs (Event ID 4104) record the **exact script/code** that was typed or run. This is extremely useful for investigators — we can literally read what the attacker typed.

Tools We Will Use

Tool	What it does	Analogy
Event Viewer	Built-in Windows app to read `.evtx` log files	Reading the CCTV notebook
PECmd (by Eric Zimmerman)	Converts Prefetch `.pf` files into readable CSV	Translating a secret code into English
Timeline Explorer (by Eric Zimmerman)	Opens CSV files, lets you filter/sort easily like Excel but built for forensics	A super-powered Excel

Basic PECmd command:

```
PECmd.exe -d "path-to-prefetch-files" --csv . --csvf outputfilename.csv
```

- `-d` → the folder containing prefetch files
- `--csv .` → save the output CSV in the current folder
- `--csvf outputfilename.csv` → name of the output file

The Scenario

Story in simple words:

A person named **Alonzo** saw weird files on his work computer and reported it. The security team (SOC) suspects a **Kerberoasting attack** happened in the network.

We are given 3 pieces of evidence:

1. **Security Logs** from the Domain Controller (DC01) — the "main office notebook"
2. **PowerShell Logs** from Alonzo's computer — "what commands were typed"
3. **Prefetch Files** from Alonzo's computer — "what programs were run"

Our job: **Connect all 3 pieces of evidence** into one story (a timeline) and answer investigation questions.

Step-by-Step Solution

🔎 Q1: When did the Kerberoasting activity happen?

Goal: Find the exact timestamp of the attack in the Domain Controller's Security logs.

How to think about it: We know Kerberoasting = requesting a TGS ticket (**Event ID 4769**) for a **real service** (not a computer account, not `krbtgt`) using **weak RC4 encryption (0x17)**.

Steps:

1. Open `SECURITY-DC.evtx` in Event Viewer.
2. Filter by **Event ID = 4769**.
3. Among the results, ignore:
   - Any service name ending in `$` (like `DC01$`) → these are just normal computer-to-computer communication, not attacks.
   - Any service named `krbtgt` → this is normal ticket-granting activity, not the target.
4. Search for **Ticket Encryption Type = 0x17** (RC4 — the weak one attackers prefer).
5. Confirm **Failure Code = 0x0** (meaning the request was SUCCESSFUL, ticket was actually issued).

The event matching ALL these conditions = the Kerberoasting event.

✅ **Answer:** `2024-05-21 03:18:09`

🔎 Q2: Which service was targeted?

How to think about it: Now that we found the exact log entry (from Q1), just **read the details** of that same event. Every 4769 event log shows a field called **"Service Name"** — this tells us exactly which service account the attacker requested a ticket for.

Steps:

1. Open the Event 4769 we found in Q1.
2. Look at "Service Information" → "Service Name".

✅ **Answer:** `MSSQLService`

🔎 Q3: What is the IP address of the attacking workstation?

How to think about it: The same event log entry also tells us **where the request came from** (like Caller ID on a phone).

Steps:

1. In the same Event 4769, look at "Network Information" → "Client Address".

✅ **Answer:** `172.17.79.129`

> 💡 **Why this matters:** Now we know WHICH computer to investigate next (Alonzo's workstation).

🔎 Q4: What tool/script was used to find Kerberoastable accounts?

How to think about it: Before an attacker can Kerberoast a specific service, they need to first **scan/enumerate** the Active Directory to find out which service accounts even exist. This scanning step usually happens BEFORE the actual attack (a few minutes earlier).

Steps:

1. Now switch to the **PowerShell-Operational logs** from the workstation (172.17.79.129).
2. Filter by **Event ID = 4104** (records full script content).
3. Look at events happening **just before** our established time (03:18:09 UTC).
4. We find:
    - First: `powershell -ep bypass` → attacker disabled PowerShell's safety restrictions (Execution Policy Bypass) so unsigned/malicious scripts can run freely.
    - Then, a huge script block appears — reading its header/comments reveals its name.

✅ **Answer:** `powerview.ps1`

> 💡 **What is PowerView?** A tool used to enumerate (list/scan) Active Directory — users, groups, computers, permissions. Attackers use it to find juicy targets, like accounts that can be Kerberoasted.

🔎 Q5: When was PowerView executed?

How to think about it: Simply check the **timestamp** of that same script block event found in Q4.

Steps:

1. Look at the "TimeCreated" → "SystemTime" field of that event in the log details (XML/Details view).

✅ **Answer:** `2024-05-21 03:16:32`

> 💡 Notice: `03:16:32` (PowerView scan) happens **before** `03:18:09` (actual Kerberoasting ticket request). This makes logical sense — recon happens before attack!

🔎 Q6: What is the exact tool (with full file path) used to actually perform the Kerberoasting attack?

How to think about it: PowerView only **finds** targets — it doesn't request the actual tickets. We need another tool for that. This is where **Prefetch files** help us — they tell us **which .exe programs ran** on the computer.

Steps:

1. Parse the prefetch files using PECmd:

```
PECmd.exe -d "path-to-prefetch-files" --csv . --csvf result.csv
```

2. Open `result.csv` in **Timeline Explorer**.
3. Filter results by date of the incident (`2024-05-21`) to reduce noise.
4. **Trick:** Look for programs where:
   - "Last Run" has a value
   - "Previous Run" is empty

   This means: "this program ran for the FIRST TIME ever on this computer" — very

suspicious for a work computer!

5. Scroll through executable names — one file called `RUBEUS.EXE` stands out (a well-known hacking tool name).
6. Check its "Last Run" time → it's `03:18:08`, just **1 second before** the DC logged the attack at `03:18:09`. This is a perfect match!
7. To get the FULL PATH of where Rubeus.exe was stored, double-click the **"Files Loaded"** column for that row. This shows every file the program touched during execution — including its own full path.

✅ **Answer:** `c:\Users\Alonzo.spire\Downloads\Rubeus.exe`

> 💡 **What is Rubeus?** A powerful tool specifically built to perform Kerberos attacks, including Kerberoasting (requesting and extracting TGS tickets for cracking).

🔎 Q7: When was the tool executed to dump credentials?

How to think about it: This is simply asking: "When did Rubeus.exe run?" — which we already found in Q6 while checking the "Last Run" column.

✅ **Answer:** `2024-05-21 03:18:08`

Final Timeline

Putting all our findings together tells the complete attack story:

Time (UTC)	Event	Meaning
`03:16:29`	`powershell -ep bypass` executed	Attacker disables PowerShell security restrictions
`03:16:32`	`powerview.ps1` script block executed	Attacker scans Active Directory to find Kerberoastable service accounts
`03:18:08`	`Rubeus.exe` executed (from `C:\Users\Alonzo.spire\Downloads\`)	Attacker runs Kerberoasting tool
`03:18:09`	Event ID `4769` logged on DC01 (Service:	Domain Controller issues the crackable TGS ticket —

	`MSSQLService`, Encryption: `0x17`)	the actual "roast" happens

In simple words:

> The attacker got access to Alonzo's computer, turned off PowerShell's safety lock, downloaded and ran a scanning tool (PowerView) to find weak service accounts, then downloaded Rubeus (a dedicated attack tool) and used it to request a crackable ticket for the MSSQL service account — all within about 2 minutes.

Summary Cheat Sheet

Key Terms

- **AD (Active Directory)** = company's user/computer database
- **DC (Domain Controller)** = server holding that database + login gatekeeper
- **Kerberos** = ticket-based login system
- **TGT** = your "wristband" after login
- **TGS** = specific "ride pass" for one service
- **Kerberoasting =** requesting TGS tickets on purpose to crack them offline and steal service passwords
- **RC4 (0x17)** = weak/old encryption type, preferred target for cracking
- **Event ID** = category code for a specific type of Windows log
- **Prefetch** = record of which programs ran, when, and what files they touched
- **PowerShell 4104 logs** = full record of scripts executed

Key Event IDs to Remember

Event ID	Meaning
4769	Kerberos Service Ticket requested
4104	PowerShell script block logged

Detection Checklist for Kerberoasting

1. Filter Event ID `4769`
2. Ignore service names ending in `$`
3. Ignore service name `krbtgt`
4. Check `Ticket Encryption Type = 0x17` (RC4)
5. Check `Failure Code = 0x0` (success)
6. Note down: Time, Service Name, Client IP, Username

Investigation Order (How Real Investigators Think)

1. Start at the Domain Controller logs → find WHEN and WHAT was targeted
2. Get the source IP → identify WHICH computer is compromised
3. Go to that computer's PowerShell logs → find recon/scanning tools used
4. Go to that computer's Prefetch files → find exact attack tools + their file paths + exact execution times
5. Build a timeline connecting everything

🎓 What You Learned

- Basics of Active Directory & Kerberos authentication
- What Kerberoasting attack really is and why RC4 encryption matters
- How Windows Event Logs and Event IDs work
- How Prefetch files help trace program execution even without antivirus alerts
- How to chain multiple types of evidence (DC logs + PowerShell logs + Prefetch) into one timeline
- Real tool usage: PECmd, Timeline Explorer, Event Viewer